02246 Mandatory Assignment
A02 - Model Checking Algorithms*

To be submitted on DTU Learn - see deadline on DTU Learn

You are encouraged to work in groups, but you must clearly identify the contributions of each group member, and you will be jointly responsible for the finished report. Register your group on DTU Learn before submitting as group submission.

Answers to all parts should be typed up using LaTeX and submitted electronically as a PDF report using the provided template. Drawings and formulae may be handwritten and scanned. More detailed instructions as to the style of answer we expect for each part are included below.

Some tasks require to upload files.

*Thanks to Michael Smith (original author), and Lijun Zhang, Kebin Zeng and Flemming Nielson and Alberto Lluch Lafuente (contributors)

A02 - Model Checking Algorithms

A02P: Practical Problems

A02P.1 - Stefano Giroletti So far we have looked at just two scheduling disciplines — FCFS and SRT. Another type of scheduler that is very widely used is the *round-robin* scheduler. Rather than running each job to completion before moving onto the next, as in the FCFS scheduler, or making the decision of which job to serve next at each time step, as in the SRT scheduler, a round-robin scheduler executes each task for just one time unit before moving onto the next, cycling through them.

a) Think about how you would model such a scheduler in PRISM, for a scenario with two clients. Using the SRT scheduler as a starting point, modify the *serve* commands, so that the scheduler executes jobs in a round-robin fashion.

You may want to approach this by first assuming that there are always two waiting jobs (i.e. job_1 and job_2 are always true), and figuring out how to cycle between them. *Hint: you will need to add some extra state to the Scheduler module.* You can then modify your solution, so that it behaves correctly when there are fewer than two jobs waiting. *Hint: look at how we shifted the queue in the FCFS scheduler.*

To implement such a scheduler, I used the SRT scheduler with two clients as a starting point. The key idea is to add a state variable to the *Scheduler* module that keeps track of the last served job. Since there are only two clients in this scenario, a simple boolean flag (e.g., *just_served1*) is sufficient.

First, consider the scenario where both *job1* and *job2* are waiting to be served:

- We allow the *serve1* operation if client 1 was not just served (i.e., the flag is set to *false*).
- We allow the *serve2* operation if client 1 was just served (i.e., the flag is set to *true*).

Naturally, during these transitions, we toggle the *just_served1* flag accordingly (setting it to *true* after serving client 1, and *false* after serving client 2).

Next, we can consider the case where only one of the two jobs is waiting. In this situation, we always serve the waiting job without checking the flag's current state. However, we must still update *just_served1* just as we did in the previous transitions, ensuring that the strict round-robin alternation resumes correctly as soon as both jobs are waiting again.

b) Make any changes that are needed to the *create* and *finish* commands, to complete the module. Explain your changes.

No changes are needed for the *create* and *finish* commands, since the flag is simply initialized to *false* in the scheduler and is exclusively modified during the *serve* commands.

UPLOAD REQUIRED: the new prism model `A02P.1.b.prism`.

c) How many states does your model have? Provide a screenshot.

There are **98** reachable states. See Figure 1.

d) Modify the properties that you have previously verified for the FCFS and SRT schedulers, and verify them for your model. Provide a screenshot showing the results.

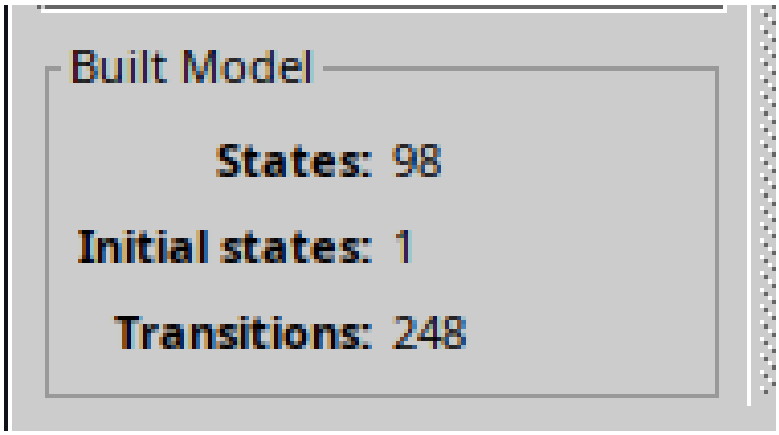


Figure 1: Number of reachable states in RR scheduler.

There is no need to edit the previous properties. In particular, we will reuse the properties from the SRT model to verify that if a client has an active job, it is correctly tracked in the queue, alongside the global properties to check for starvation. Since we need to verify this for both clients, we have a total of four properties:

- `A [ G (state1=1 => job1=true) ]`: verifies the queueing logic for client 1 (see Figure 2).
- `A [ G (state2=1 => job2=true) ]`: verifies the queueing logic for client 2 (see Figure 3).
- `P>=1 [ G task1>0 => P>=1 [ F task1=0 ] ]`: verifies the absence of starvation for client 1 (see Figure 4).
- `P>=1 [ G task2>0 => P>=1 [ F task2=0 ] ]`: verifies the absence of starvation for client 2 (see Figure 5).

The evaluation results for these properties on the model are shown below.

UPLOAD REQUIRED: the new prism properties `A02P.1.d.props`.

e) If a property fails to hold, check whether this was due to a mistake in your model, or is a real problem with the round-robin scheduler. In the case of the former, explain what the mistake was, and fix it in your model. In case of the latter, explain what the problem is.

None of the checks failed: if either client 1 or client 2 has an active job, it is correctly maintained in the queue. Furthermore, thanks to the round-robin policy, both jobs are guaranteed to eventually complete without starvation, meaning all verified properties hold true.

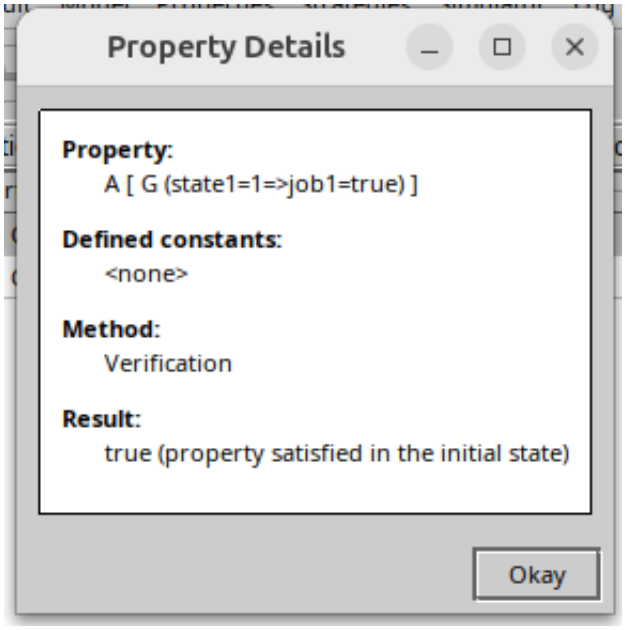


Figure 2: Queueing property for client 1.

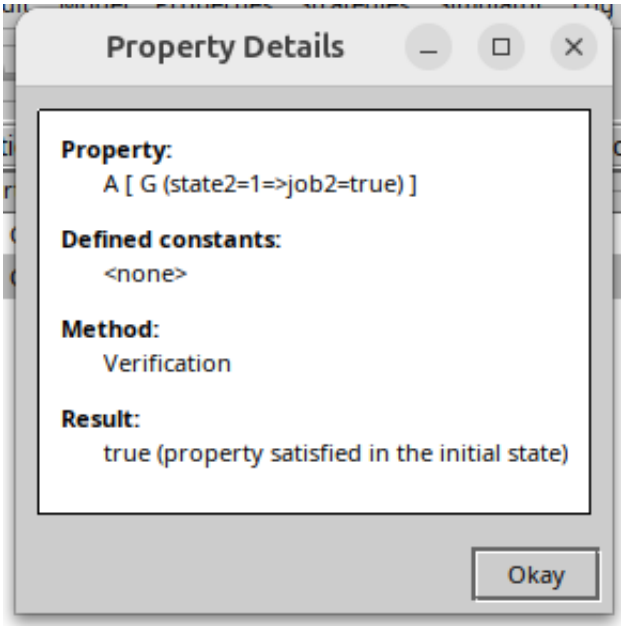


Figure 3: Queueing property for client 2.

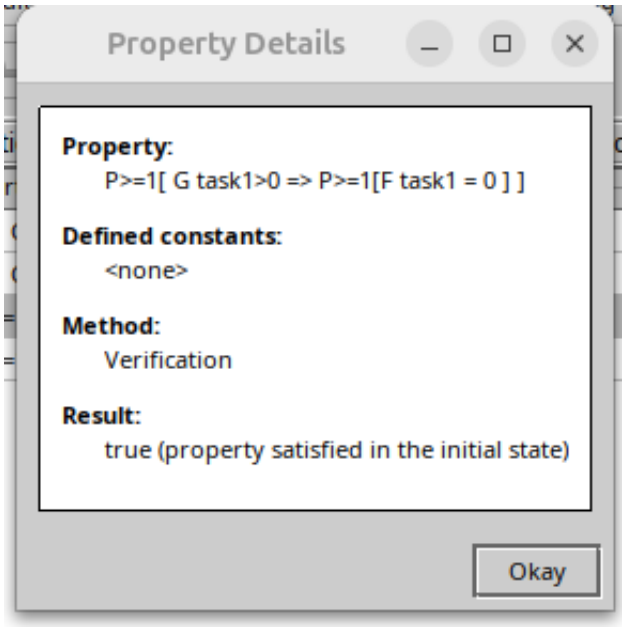


Figure 4: Starvation property for client 1.

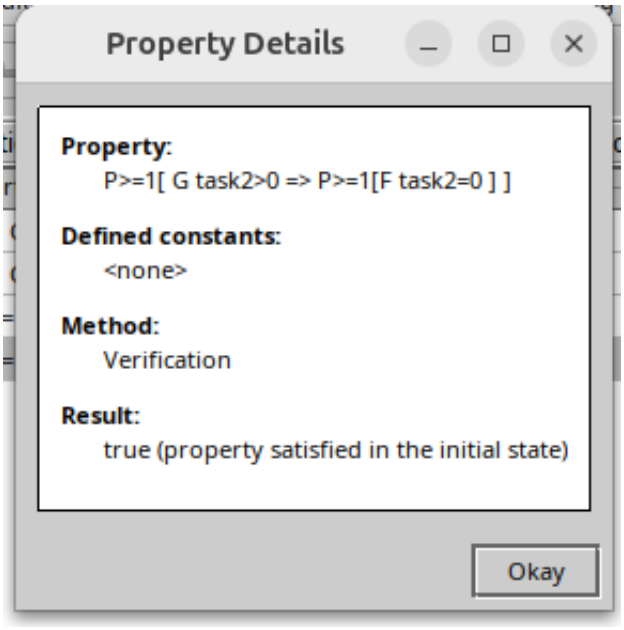


Figure 5: Starvation property for client 2.

A02P.2 - Maria Mitsi In many real schedulers, we need to take into account that tasks have different *priorities*. For example, operating system processes for tasks such as memory management and hardware control need to be given priority over user level processes such as a word processor or web browser. Your challenge is to extend the FCFS server to handle tasks with two different priority levels.

a) Modify the *Client* module so that it can create tasks with two different priority levels. You need to decide how to encode the priorities — do you use an additional variable, or do you encode it in the actions? (note: there is not only one correct answer here). Explain the main ide of your new model.

I encode the priority in the action used to create a job. The actions `create1l` and `create1h` create low- and high-priority jobs, respectively. The scheduler can therefore determine the priority of a new job directly from the action.

The job length is still chosen non-deterministically between 1 and 5, as in the original FCFS model. (See below for the client 1 & 2 modifications)

```
module client1
  state1 : [0..1] init 0; // State of the job (inactive/active)
  task1 : [0..5] init 0; // Length of the job

  // Create a low-priority job
  [create1l] state1=0 -> (state1'=1) & (task1'=1);
  [create1l] state1=0 -> (state1'=1) & (task1'=2);
  [create1l] state1=0 -> (state1'=1) & (task1'=3);
  [create1l] state1=0 -> (state1'=1) & (task1'=4);
  [create1l] state1=0 -> (state1'=1) & (task1'=5);

  // Create a high-priority job
  [create1h] state1=0 -> (state1'=1) & (task1'=1);
  [create1h] state1=0 -> (state1'=1) & (task1'=2);
  [create1h] state1=0 -> (state1'=1) & (task1'=3);
  [create1h] state1=0 -> (state1'=1) & (task1'=4);
  [create1h] state1=0 -> (state1'=1) & (task1'=5);

  // Serve the job
  [serve1] state1=1 & task1>0 -> (task1'=task1-1);

  // Complete the job
  [finish1] state1=1 & task1=0 -> (state1'=0);

endmodule

module client2 = client1 [state1=state2,
                          task1=task2,
                          create1l=create2l,
                          create1h=create2h,
                          serve1=serve2,
                          finish1=finish2 ]
endmodule
```

b) Modify the *Scheduler* module so that when a new job arrives, it moves ahead of any lower priority jobs that are in the queue — this may involve pre-empting (interrupting the execution of) the currently running job so that the higher priority job can run. Note that how you do this will depend on the choices you made in modifying the *Client* module. Explain your solution.

The scheduler stores the client and priority of each job in the variables `job1` and `job2`. A high-priority job is placed before a low-priority job. If a high-priority job arrives while a low-priority job is running, the low-priority job is moved to `job2` and the new job becomes `job1`. This implements pre-emption.

The complete model is provided in `A02P.2.b.prism`. Below is the code snippet of the scheduler.

```
module scheduler
  // Encoding:
  // 0 = empty
  // 1 = client 1, low priority
  // 2 = client 1, high priority
  // 3 = client 2, low priority
  // 4 = client 2, high priority

  job1 : [0..4] init 0; // Running/head-of-queue job
  job2 : [0..4] init 0; // Waiting job

  // ------------------------------------------------
  // Client 1 creates a low-priority job
  // ------------------------------------------------

  // Empty queue
  [create1l] job1=0 -> (job1'=1);

  // Non-empty queue: low priority goes behind the current job
  [create1l] job1>0 & job2=0 -> (job2'=1);


  // ------------------------------------------------
  // Client 1 creates a high-priority job
  // ------------------------------------------------

  // Empty queue
  [create1h] job1=0 -> (job1'=2);

  // Pre-empt a low-priority job
  [create1h] job1=1 & job2=0 -> (job1'=2) & (job2'=1);
  [create1h] job1=3 & job2=0 -> (job1'=2) & (job2'=3);

  // High-priority job behind another high-priority job
  [create1h] job1=2 & job2=0 -> (job2'=2);
  [create1h] job1=4 & job2=0 -> (job2'=2);
```

```
// ------------------------------------------------
// Client 2 creates a low-priority job
// ------------------------------------------------

// Empty queue
[create2l] job1=0 -> (job1'=3);

// Non-empty queue: low priority goes behind current job
[create2l] job1>0 & job2=0 -> (job2'=3);


// ------------------------------------------------
// Client 2 creates a high-priority job
// ------------------------------------------------

// Empty queue
[create2h] job1=0 -> (job1'=4);

// Pre-empt a low-priority job
[create2h] job1=1 & job2=0 -> (job1'=4) & (job2'=1);
[create2h] job1=3 & job2=0 -> (job1'=4) & (job2'=3);

// High-priority job behind another high-priority job
[create2h] job1=2 & job2=0 -> (job2'=4);
[create2h] job1=4 & job2=0 -> (job2'=4);


// ------------------------------------------------
// Serve the job at the head of the queue
// ------------------------------------------------

[serve1] job1=1 -> true;
[serve1] job1=2 -> true;

[serve2] job1=3 -> true;
[serve2] job1=4 -> true;


// ------------------------------------------------
// Complete the job at the head of the queue
// ------------------------------------------------

[finish1] job1=1 -> (job1'=0);
[finish1] job1=2 -> (job1'=0);

[finish2] job1=3 -> (job1'=0);
[finish2] job1=4 -> (job1'=0);


// Move the waiting job to the head
```

```
  [] job1=0 & job2>0 -> (job1'=job2) & (job2'=0);

endmodule
```

c) How many states does your model have? Provide a screenshot.
As shown in Figure 6, the model contains 311 reachable states.

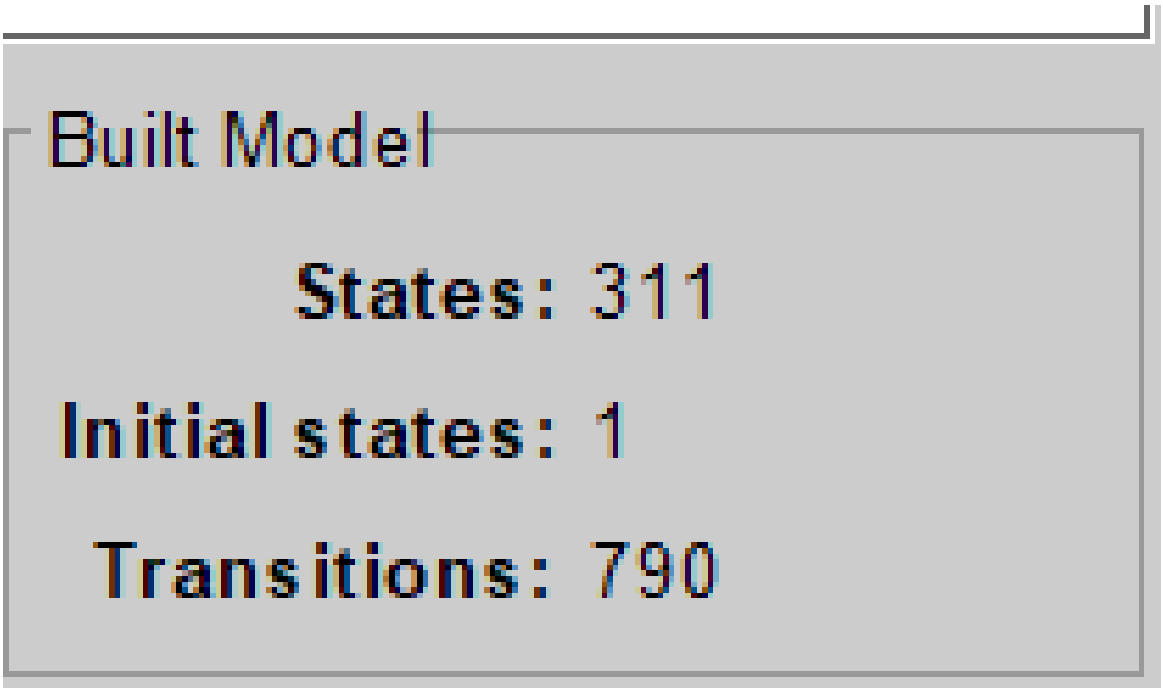


Figure 6: PRISM state-space results for A02.P2.c.

d) Modify the properties that you have previously verified for the FCFS schedulers, and verify them for your model. Provide a screenshot showing the results. The properties were modified to distinguish between high-priority jobs from client 1 and client 2. The scheduler encodes a high-priority job from client 1 as 2 and a high-priority job from client 2 as 4, in either the head of the queue (`job1`) or the waiting position (`job2`).
The first two properties are satisfied, meaning that a high-priority job from either client eventually finishes with probability 1. The two global properties are not satisfied. The verification results are shown below.

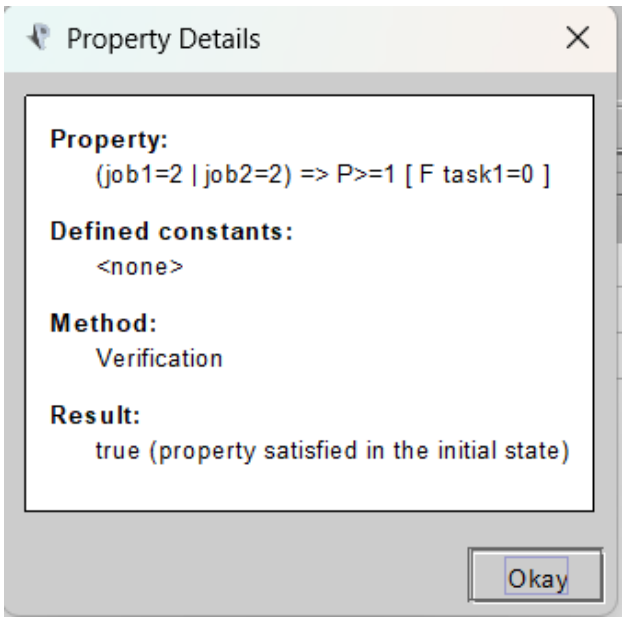


Figure 7: Property 1

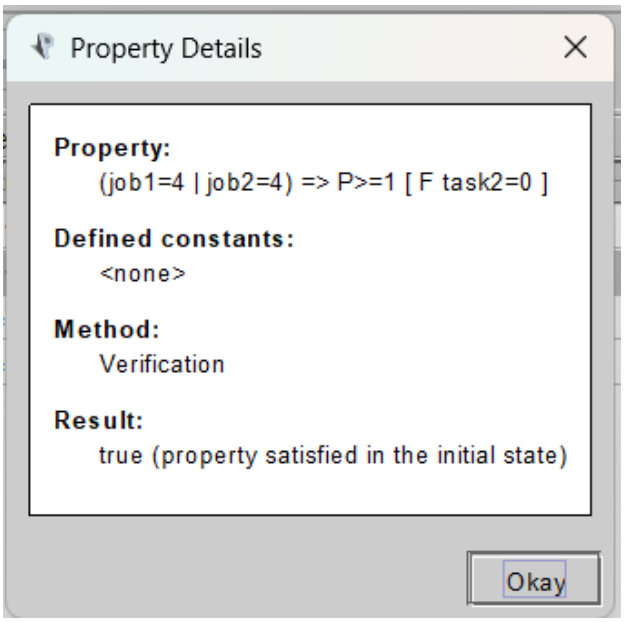


Figure 8: Property 2

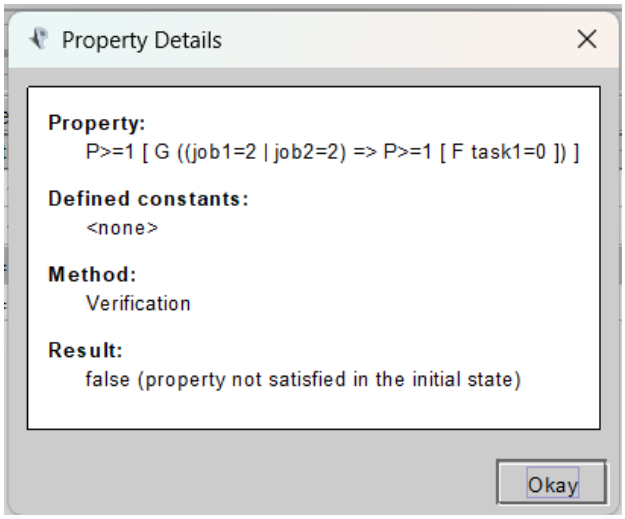


Figure 9: Property 3

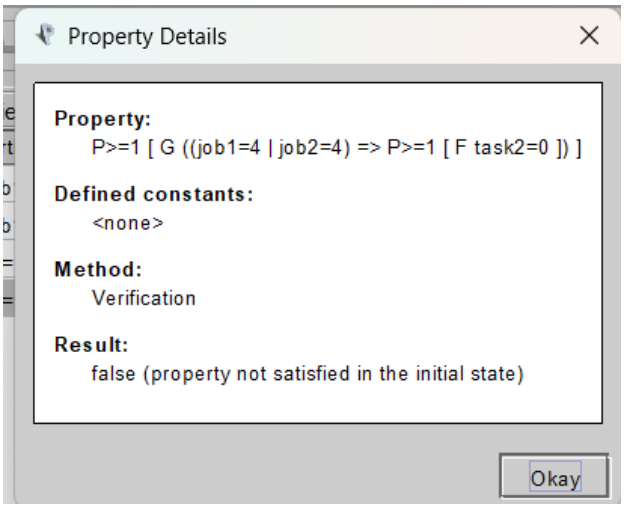


Figure 10: Property 4

e) If a property fails to hold, check whether this was due to a mistake in your model, or is a real problem with the priority scheduler. In the case of the former, explain what the mistake was, and fix it in your model. In case of the latter, explain what the problem is.

The two global properties did not hold in the initial model. After examining the model, we found that this was caused by a modelling mistake rather than a limitation of the priority scheduler.

The problem was in the rules for creating a new job when the queue was considered empty. These rules only checked whether `job1=0`, without checking whether a job was still waiting in `job2`. Consequently, a new job could be created while a previously waiting job was still stored in `job2`. This could overwrite the waiting position and cause a high-priority job to be postponed indefinitely.

We fixed the model by requiring both queue positions to be empty before a new job can be placed directly in `job1`:

```
[create1l] job1=0 & job2=0 -> (job1'=1);
[create1h] job1=0 & job2=0 -> (job1'=2);
[create2l] job1=0 & job2=0 -> (job1'=3);
[create2h] job1=0 & job2=0 -> (job1'=4);
```

After this correction, when `job1=0` and a job is waiting in `job2`, the scheduler can only move the waiting job to the head of the queue. This prevents waiting jobs from being overwritten and ensures that the priority mechanism is preserved.We can see the verification of each property below.

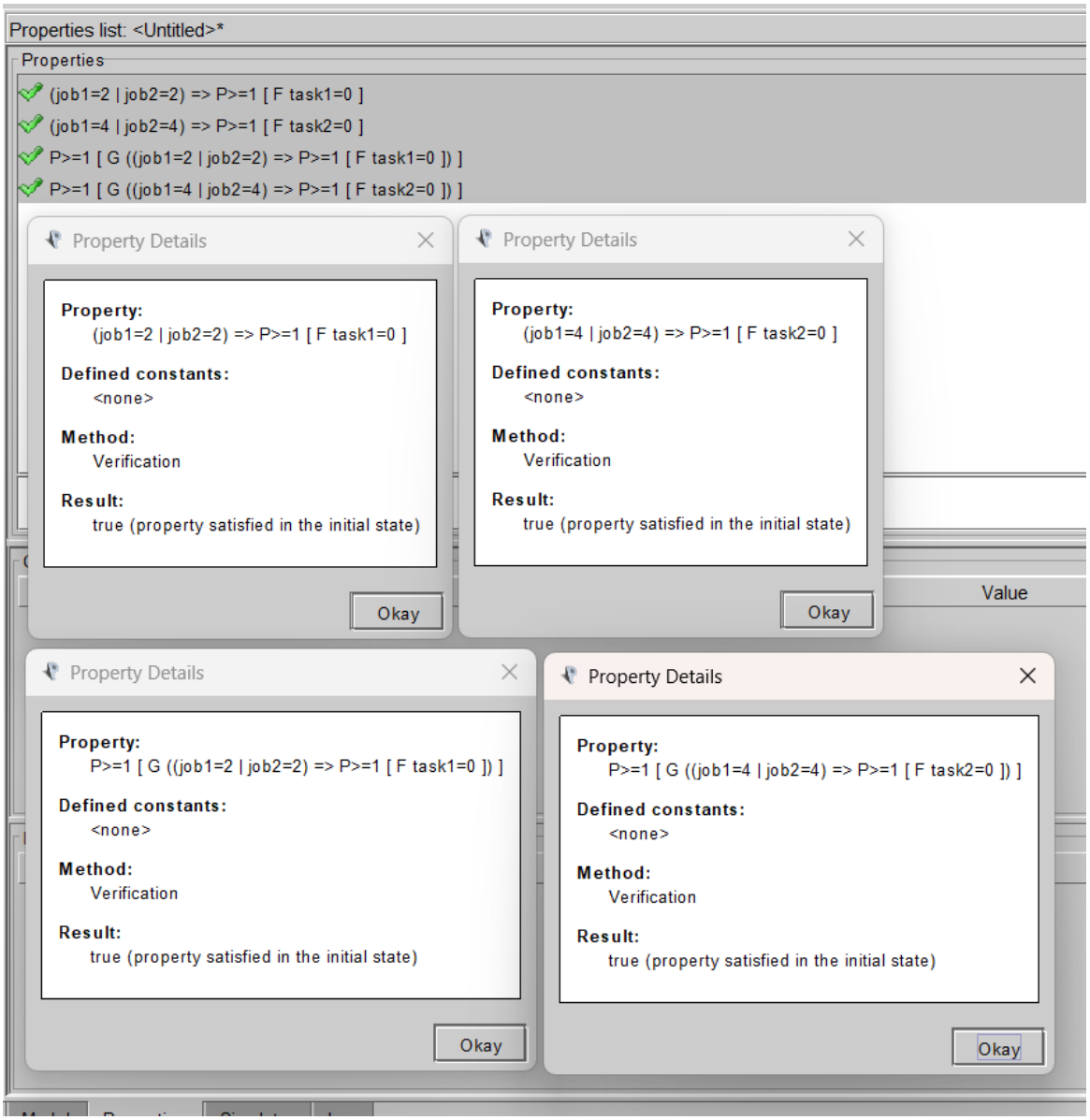


Figure 11: Verified properties after modification

A02: Theoretical Problems

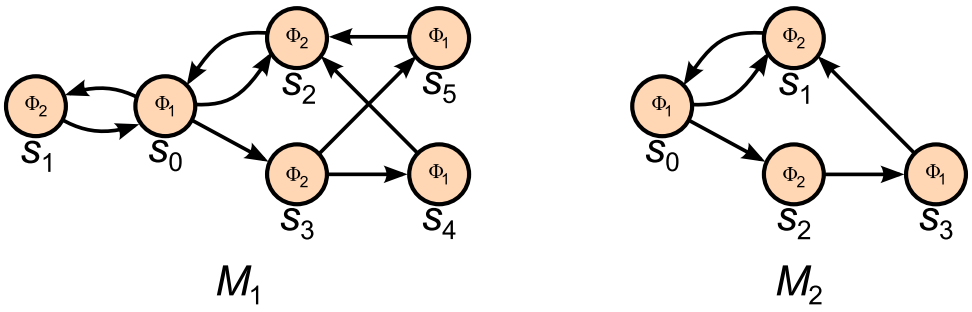


Figure 12: Two transition systems, M_1 and M_2

A02T.1 Consider the CTL formula $\Phi = \neg(EX\ \Phi_1) \wedge (AF\ \Phi_2)$, where Φ_1 and Φ_2 are atomic propositions. We will apply the model checking algorithm, described in Section 6.4 of the book.

a) Convert Φ into $\exists$-normal form, and draw the abstract syntax tree of the resulting formula, which we will call Φ'.

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

b) Compute the set of satisfying states $Sat(\Phi) = Sat(\Phi')$ for the transition system M_1 from Figure 12, explaining how you do this, including how you computed the satisfaction set of each sub-formula.

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

A02T.2 Design algorithms to the compute below satisfaction sets directly (without converting them into ECTL). For each case, sketch the algorithm as pseudo-code (as in the slides) and argue for its correctness.

HINT: Start by having a look at the algorithms seen in class for the existential variants of the below formulas. Ask yourself what you would change to make them work for universal quantification ($\forall$) instead of existential quantification ($\exists$). When you provide your answer, you will likely want to highlight (with some color) the difference between both cases.

a) $sat(\forall(\mathsf{X}\phi))$

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

b) $sat(\forall(\phi_1 \mathsf{U} \phi_2))$

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

c) $sat(\forall\Box\phi)$

Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

A02T.3

1. Consider the two transition systems in Figure 12: $M_1 = (S_1, \rightarrow_1, \{ s_0 \}, AP, L_1)$ and $M_2 = (S_2, \rightarrow_2, \{ s_0 \}, AP, L_2)$, where $AP = \{ \Phi_1, \Phi_2 \}$.

   (a) Write down the coarsest (i.e. the largest) bisimulation relation you can find between the states of the two structures: $R_{12} \subseteq S_1 \times S_2$.
   Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

   (b) Argue why R_{12} is a bisimulation, and argue why it is maximal.
   Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

   (c) Write down the coarsest bisimulation relation you can find between the states of M_1: $R_{11} \subseteq S_1 \times S_1$. For conciseness, provide the relation as a partition of the states (i.e. as the set of equivalence classes), instead of enumerating all pairs in the relation.
   Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

   (d) Argue why R_{11} is a bisimulation, and argue why it is maximal.
   Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

   (e) Construct the bisimulation quotient $M_1/\sim$ of M_1.
   Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.

   (f) Is the quotient bisimilar to M_2? If so, what is the largest bisimulation relation between them?
   Provide your answer here. Leave the special color (blue). Figures, tables, code snippets can be placed somewhere else but they need to be referred here.